

binary_search 정리

정렬된 범위에 특정 값이 존재하는지 빠르게 확인하는 알고리즘입니다.

사용처 (Where)

- 정렬된 vector에서 존재 여부만 확인
- 검색을 여러 번 반복할 때
- 범위가 크고 빠른 검색이 필요할 때

방법 (How to Use)

- 먼저 정렬
- begin/end와 찾을 값을 넘김
- 반환값은 bool
- 위치가 필요하다면 lower_bound 사용

기본 정보

- 필요 헤더: `#include <algorithm>`
- 기본 선언: `bool ok = binary_search(v.begin(), v.end(), x);`

왜 써야 할까? (Why)

- 정렬된 큰 데이터에서 빠른 존재 확인 가능
- 직접 이분 탐색보다 코드가 간단
- bool 결과만 필요할 때 적합

주요 함수

함수/문법	의미
<code>binary_search</code>	존재 여부
<code>sort</code>	사전 정렬
<code>lower_bound</code>	위치 확인
<code>find</code>	선형 검색

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {5, 1, 3, 2};
    sort(v.begin(), v.end());

    cout << binary_search(v.begin(), v.end(), 3);
}
```

binary_search 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
binary_search	$O(\log n)$
정렬	$O(n \log n)$
find	$O(n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	binary_search 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 회원 번호 존재 확인

회원 번호 목록을 정렬한 뒤 특정 번호가 있는지 빠르게 확인합니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> ids = {1003, 1001, 1007, 1002};
    sort(ids.begin(), ids.end());

    int target = 1007;
    if (binary_search(ids.begin(), ids.end(), target)) {
        cout << "registered";
    }
}
```

처음 배울 때 자주 하는 실수

- 정렬 전 사용 금지
- 위치는 알 수 없음
- 삽입 위치나 개수가 필요하면 lower_bound/upper_bound 사용

비슷한 항목과 차이

- lower_bound는 위치 반환
- find는 정렬 없이 사용 가능하지만 느림
- unordered_set은 정렬 없이 평균 $O(1)$ 존재 확인

한 줄 정리: `binary_search`은(는) '정렬된 `vector`에서 존재 여부만 확인' 상황에서 먼저 떠올리면 좋은 STL입니다.